I need a Python function called “split_by_time(df: pd.DataFrame, test_ratio: float = 0.2) → tuple” that splits a prefix DataFrame into train and test sets based on case order. The function should: find all unique case IDs in the 'case_id' column; take the last “test_ratio” fraction of unique case IDs as test set and the rest as train set (preserving original order, no sorting needed); filter the DataFrame to return (train_df, test_df) as a tuple. This ensures test cases come from later cases in the log

LISA Flash

Here is the Python function that performs a time-based split on a DataFrame based on unique `case_id` occurrences, ensuring that the test set consists of the most recent cases while preserving the original order of rows within each set.

python Свернуть Запустить Копировать

```python
import pandas as pd

def split_by_time(df: pd.DataFrame, test_ratio: float = 0.2) -> tuple:
    """
    Splits a prefix DataFrame into train and test sets based on case order.

    The function identifies unique case IDs in the order they first appear,
    assigns the last 'test_ratio' fraction of these unique IDs to the test set,
    and the rest to the train set. It preserves the original row order.

    Parameters:
    df (pd.DataFrame): Input DataFrame with a 'case_id' column.
    test_ratio (float): Fraction of unique case IDs to include in the test set
(default 0.2).

    Returns:
    tuple: (train_df, test_df) where both are DataFrames.
    """
    if not isinstance(df, pd.DataFrame):
        raise TypeError("Input must be a pandas DataFrame.")
    if 'case_id' not in df.columns:
        raise ValueError("DataFrame must contain a 'case_id' column.")
    if not 0 < test_ratio < 1:
        raise ValueError("test_ratio must be between 0 and 1 (exclusive).")

    # Get unique case IDs in the order of their first appearance
    unique_case_ids = df['case_id'].drop_duplicates().tolist()
    total_unique_cases = len(unique_case_ids)
```

```
    if total_unique_cases == 0:
        return df.copy(), pd.DataFrame(columns=df.columns)

    # Calculate the cutoff index for splitting
    # We want the last N cases for test, where N = ceil(test_ratio * total)
    # Or strictly floor/ceil depending on desired strictness.
    # Usually, we calculate the split point from the start.
    test_count = int(total_unique_cases * test_ratio)

    # Ensure at least 1 item goes to test if possible, otherwise handle edge cases
    if test_count == 0 and total_unique_cases > 0:
        test_count = 1

    # Split the list of unique IDs
    train_ids_set = set(unique_case_ids[:-test_count]) if test_count > 0 else
set(unique_case_ids)
    test_ids_set = set(unique_case_ids[-test_count:]) if test_count > 0 else set()

    # Filter the original DataFrame based on these sets
    # Using .isin() preserves the original row order
    train_df = df[df['case_id'].isin(train_ids_set)]
    test_df = df[df['case_id'].isin(test_ids_set)]

    return train_df, test_df
```

How it works:

Identify Unique Cases: It extracts `case_id`s using `drop_duplicates()`, which keeps the order of first appearance. This represents the chronological order of cases in the log.

Calculate Split: It calculates how many unique IDs belong to the test set based on `test_ratio`.

Assign Sets: It splits the list of unique IDs into a "train" portion (all but the last few) and a "test" portion (the last few).

Filter: It filters the original DataFrame using `.isin()` against these ID sets. Since the original DataFrame isn't sorted or modified, the resulting `train_df` and `test_df` maintain the exact row order from the input.

> I need a Python function called “save_results(results_df: pd.DataFrame, model_name: str, dataset_name: str, output_dir: str)” that takes a DataFrame with columns 'prefix_length', 'accuracy', 'mae', and saves it as a CSV file to output_dir with filename formatted as '{model_name}_{dataset_name}_results.csv'. Create the output directory if it does not exist using os.makedirs

LISA Flash

Here is the Python function to save your results. It includes the necessary imports, handles directory creation, and formats the filename as requested.

python Свернуть Запустить Копировать

```python
import os
import pandas as pd

def save_results(results_df: pd.DataFrame, model_name: str, dataset_name: str, output_dir: str) -> None:
    """
    Saves a DataFrame containing evaluation metrics to a CSV file.

    Parameters:
    results_df (pd.DataFrame): DataFrame with columns 'prefix_length', 'accuracy', 'mae'.
    model_name (str): Name of the model used for the filename.
    dataset_name (str): Name of the dataset used for the filename.
    output_dir (str): Path to the directory where the file will be saved.

    Raises:
    ValueError: If required columns are missing in the DataFrame.
    """
    # Validate that required columns exist
    required_columns = ['prefix_length', 'accuracy', 'mae']
    if not all(col in results_df.columns for col in required_columns):
        raise ValueError(f"DataFrame must contain columns: {required_columns}")

    # Create the output directory if it doesn't exist
    os.makedirs(output_dir, exist_ok=True)

    # Construct the filename
    filename = f"{model_name}_{dataset_name}_results.csv"
    filepath = os.path.join(output_dir, filename)

    # Save to CSV
    results_df.to_csv(filepath, index=False)

    print(f"Results saved successfully to: {filepath}")
```

I need a Python function called “plot_comparison(results_dict: dict, metric: str, dataset_name:

str, output_dir: str)" where results_dict is a dictionary mapping model names (strings) to DataFrames with columns 'prefix_length' and the given metric column name. The function should create a line plot using matplotlib with prefix_length on the x-axis and the metric on the y-axis, one line per model, with a legend, axis labels, and title formatted as '{metric} by Prefix Length — {dataset_name}'. Save the figure as PNG to output_dir with filename '{metric}_{dataset_name}.png'. Create output_dir if it does not exist

LISA Flash

Here is the Python function to generate and save the comparison plot. It handles directory creation, validates inputs, and configures the matplotlib figure with the requested labels and title.

python Свернуть Запустить Копировать

```python
import os
import matplotlib.pyplot as plt
import pandas as pd

def plot_comparison(results_dict: dict, metric: str, dataset_name: str, output_dir:
str) -> None:
    """
    Creates a line plot comparing model performance across prefix lengths.

    Parameters:
    results_dict (dict): Dictionary mapping model names (str) to DataFrames.
                         Each DataFrame must have 'prefix_length' and the specified
'metric' column.
    metric (str): The name of the metric column to plot on the y-axis.
    dataset_name (str): Name of the dataset used for the plot title.
    output_dir (str): Directory path where the plot will be saved.

    Raises:
    ValueError: If required columns are missing in any DataFrame or if the
dictionary is empty.
    """
    if not results_dict:
        raise ValueError("results_dict cannot be empty.")

    # Ensure output directory exists
    os.makedirs(output_dir, exist_ok=True)

    plt.figure(figsize=(10, 6))

    for model_name, df in results_dict.items():
```

```
        # Validate columns for this specific model's DataFrame
        if 'prefix_length' not in df.columns:
            raise ValueError(f"DataFrame for model '{model_name}' is missing
'prefix_length' column.")
        if metric not in df.columns:
            raise ValueError(f"DataFrame for model '{model_name}' is missing
'{metric}' column.")

        # Sort by prefix_length to ensure lines connect correctly if data is
unsorted
        df_sorted = df.sort_values('prefix_length')

        plt.plot(
            df_sorted['prefix_length'],
            df_sorted[metric],
            marker='o',
            label=model_name
        )

    # Formatting the plot
    plt.title(f"{metric} by Prefix Length - {dataset_name}")
    plt.xlabel("Prefix Length")
    plt.ylabel(metric.capitalize())
    plt.legend()
    plt.grid(True, linestyle='--', alpha=0.7)

    # Construct filename and save
    filename = f"{metric}_{dataset_name}.png"
    filepath = os.path.join(output_dir, filename)

    plt.savefig(filepath, dpi=300, bbox_inches='tight')
    plt.close() # Close the figure to free memory

    print(f"Plot saved successfully to: {filepath}")
```